

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Математическая логика и теория формальных языков
Отчёт о выполнении лабораторной работы №1
«Проверка корректности модели автомобиля марки ВАЗ,
УАЗ, ГАЗ и Москвич через реализацию конечного автомата
для регулярного выражения»

Вариант 35

Студент,

группа 5130201/30102

_____ Путята М. А.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Регулярное выражение	4
1.2 Регулярное выражение используемое в работе	5
1.3 Конечный автомат	6
1.4 Недетерминированный конечный автомат	6
1.5 Детерминированный конечный автомат	11
2 Особенности реализации	17
2.1 Структуры данных	17
2.1.1 Класс Action	17
2.1.2 Класс FSM	18
2.2 Функция обработки пользовательского ввода	29
2.3 Создание конечного автомата соответствующего заданному регулярно- му выражению	33
3 Результаты работы программы	35
Заключение	38
Список использованной литературы	40
Приложение А. Исходный код создания конечных автоматов	41

Введение

Целью данной работы является разработка программы, которая сможет определять корректность введенных пользователем строк и генерировать новые корректные строки, используя детерминированный конечный автомат.

Задание: По заданному варианту построить регулярное выражение, затем недетерминированный конечный автомат и детерминировать его (переходы можно задавать диапазонами). Реализовать программу, которая проверяет введенный текст через реализацию конечного автомата (варианты вывода: строка соответствует, не соответствует, символы не из алфавита). Также необходимо реализовать функцию случайной генерации верной строки по полученному конечному автомату.

Вариант 35: Проверка правильно введенной марки автомобиля ВАЗ, УАЗ, ГАЗ, Москвич (с учетом старых моделей).

Для выполнения поставленной задачи необходимо:

1. Построить регулярное выражение, соответствующее варианту.
2. По полученному регулярному выражению построить недетерминированный конечный автомат.
3. Детерминировать конечный автомат.
4. Создать программную реализацию конечного автомата.
5. Создать инструменты для проверки корректности строк и генерации корректных строк.

1 Математическое описание

1.1 Регулярное выражение

Регулярное выражение – способ описания регулярных языков над конечным словарём Σ .

Регулярное выражение состоит из:

- базовых элементов:

- $\emptyset$ – пустое множество символов;
- ϵ – пустая цепочка символов;
- $a \in \Sigma$ – любой символ из словаря Σ ;

- операций:

- **Конкатенация** $R_1 R_2$ – множество цепочек, образованных соединением цепочек R_1 и R_2 .
- **Объединение** $R_1 + R_2$ или $R_1 | R_2$ – множество цепочек, принадлежащих R_1 или R_2 .
- **Итерация Клини** R^* – множество цепочек, состоящих из нуля и более повторений цепочки R .
- **Усеченная итерация** R^+ – множество цепочек, состоящих из 1 и более повторений цепочки R .

Приоритеты операций:

1. Итерация Клини.
2. Усечённая итерация.
3. Конкатенация.
4. Объединение.

Для изменение порядка выполнения операций можно использовать круглые скобки «(» и «)».

Регулярное множество – это бесконечное множество цепочек построенных, построенных из символов конечного словаря с использованием вышеперечисленных операций.

Например, регулярное выражение « $a^*b + cd^+$ » будет задавать следующее множество:

$$\{b, ab, aab, aaab, \dots\} \cup \{cd, cdd, cddd, \dots\}$$

Регулярный язык – подмножество Σ^* , которое можно описать с помощью регулярного выражения.

Операции над регулярными языками

Пусть L_1 и $L_2 \subseteq \Sigma^*$, тогда для данных языков определены следующие операции.

- **Конкатенация:** $L_1 \cdot L_2 = \{w_1w_2 \mid w_1 \in L_1 \wedge w_2 \in L_2\}$.
Пример: $\{a, b\} \cdot \{b, c\} = \{ab, ac, bb, bc\}$
- **Объединение:** $L_1 \cup L_2 = \{w \mid w \in L_1 \vee w \in L_2\}$.
Пример: $\{a, b\} \cup \{b, c\} = \{a, b, c\}$
- **Итерация Клини:** $L^* = \bigcup_{k=0}^{\infty} L^k$, где $L^{k+1} = L^k \cdot L$, $L^0 = \{\epsilon\}$.
Пример: $\{a, b\}^* = \{\epsilon, a, aa, ab, baaaba, \dots\}$
- **Усечённая итерация:** $L^* = \bigcup_{k=1}^{\infty} L^k$
Пример: $\{a, b\}^* = \{a, aa, ab, baaaba, \dots\}$

Регулярные языки, реализуемые через регулярные выражения, лежат в основе обработки текстовой информации, поиска шаблонов, валидации данных и лексического анализа.

1.2 Регулярное выражение используемое в работе

Для определения корректности введённой модели автомобиля было создано 4 регулярных выражения для каждой рассматриваемой марки автомобиля:

- регулярное выражения для моделей автомобилей марки ГАЗ:
«ГАЗ (1[2348]|2([1245]|(3(08|30))))|31(0(2(21?|9)?|[35])|1(05?|1))|46|6[1479]|А|М-(1|20|72)|Siber)»
- регулярное выражение для моделей автомобилей марки УАЗ:
«УАЗ (2(206|3(15|6(08|[25]|32))|7(46|60|722)|9232)|3(1(5(0|1[249]?|3|9)|6[025])|303|741|9(09[45]?|625?))|469|Cargo|Combi|Hunter|Patriot|Pickup|Profi)»
- регулярное выражение для моделей автомобилей марки Москвич:
«Москвич (21(3[678]|4[012])|3|4(0[0-37-8]|1[012]|2[3467]|3(0|4П))|[568]|Дуэт|Иван Калита|Князь Владимир|Святогор|Юрий Долгорукий)»
- регулярное выражение для моделей автомобилей марки ВАЗ:
«(ВАЗ|LADA) (1111|2(1(0[1-9]|1[0-5]|2[0139]|31)|32[89])|Aura|EL Lada|Granta|Iskra|Kalina|Largus|Niva|Priora|Revolution|Vesta|XRAY)»

Далее данные регулярные выражение были объединены с помощью трёх операторов «|» и было получено итоговое регулярное выражение для всех четырёх марок автомобилей:

«ГАЗ (1[2348]|2([1245]|(3(08|30))))|31(0(2(21?|9)?|[35])|1(05?|1))|46|6[1479]|А|М-(1|20|72)|Siber)|УАЗ (2(206|3(15|6(08|[25]|32))|7(46|60|722)|9232)|3(1(5(0|1[249]?|3|9)|6[025])|303|741|9(09[45]?|625?))|469|Cargo|Combi|Hunter|Patriot|Pickup|Profi)|Москвич (21(3[678]|4[012])|3|4(0[0-37-8]|1[012]|2[3467]|3(0|4П))|[568]|Дуэт|Иван Калита|Князь Владимир|Святогор|Юрий Долгорукий)|(ВАЗ|LADA) (1111|2(1(0[1-9]|1[0-5]|2[0139]|31)|32[89])|Aura|EL Lada|Granta|Iskra|Kalina|Largus|Niva|Priora|Revolution|Vesta|XRAY)»

1.3 Конечный автомат

Конечный автомат – математическая модель, определяющаяся набором $A = (S, \Sigma, Y, S_0, \delta, \lambda)$, где:

- $S = \{ S_0 \} \cup \{ VAZ1, \dots, VAZ3 \} \cup \{ LADA1, \dots, LADA4 \} \cup \{ V0, \dots, V18 \} \cup \{ MOS1, \dots, MOS20 \} \cup \{ M0, \dots, M20 \} \cup \{ GAZ1, \dots, GAZ3 \} \cup \{ G0, \dots, G33 \} \cup \{ UAZ1, \dots, UAZ3 \} \cup \{ U1, \dots, U52 \}$ – конечное множество состояний;
- $\Sigma = \{ \text{”_”}, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, C, D, E, G, H, I, K, L, M, N, P, R, S, V, X, Y, a, b, c, d, e, f, g, i, k, l, m, n, o, p, r, s, t, u, v, A, B, \Gamma, \Delta, \Xi, \text{И}, K, M, \Pi, C, \Psi, \text{Ю}, a, b, \Gamma, \Delta, \Xi, \text{и}, \text{й}, k, l, m, n, o, p, c, t, y, \text{ч}, \text{ь}, \text{э}, \text{я} \}$ – конечное множество входных сигналов;
- $Y = \{ \}$ – конечное множество выходных сигналов;
- S_0 – начальное состояние, при этом $S_0 \in S$;
- $\delta : S \times \Sigma \rightarrow S$ – функция переходов;
- $\lambda : S \times \Sigma \rightarrow Y$ – функция выходов;

Различают недетерминированные (НДКА) и детерминированные конечные автоматы (ДКА). Основным отличием НДКА от ДКА является наличие ϵ -переходов и возможность неоднозначных переходов.

1.4 Недетерминированный конечный автомат

Недетерминированные конечные автоматы крайне удобны для создания распознавателей, так как:

- обычно имеют меньше состояний, чем детерминированные конечные автоматы;
- наличие ϵ -переходов упрощает создание композиций из НДКА;
- регулярные выражения легко преобразуются в НДКА.

По полученным регулярным выражениям было построено 4 НДКА, каждый из которых соответствует определённой марке автомобиля. Данные НДКА представлены на рисунках 1 - 4.

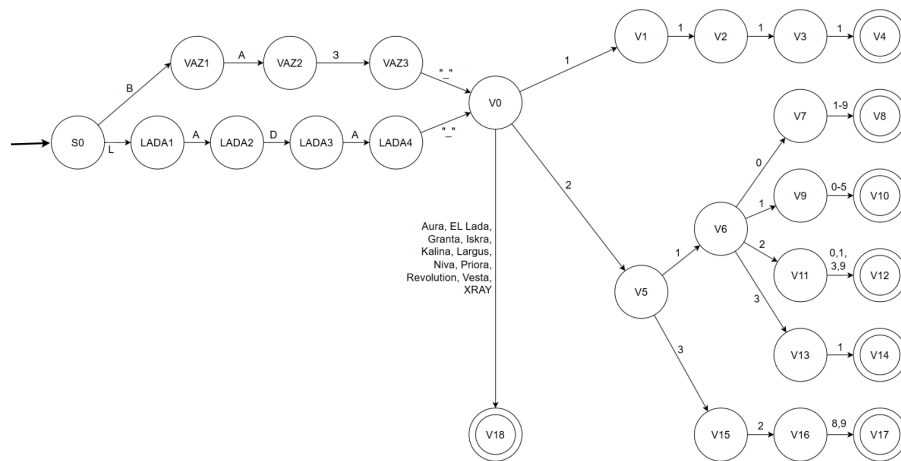


Рис. 1. НДКА для ВАЗ

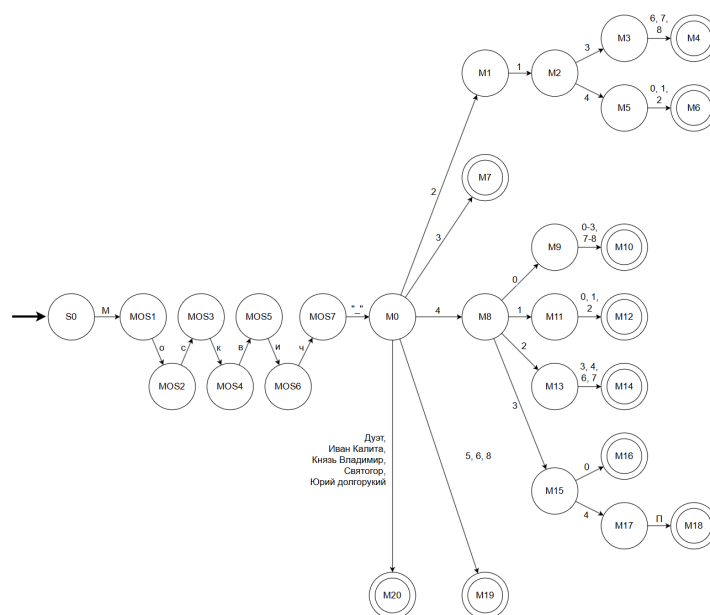


Рис. 2. НДКА для Москвич

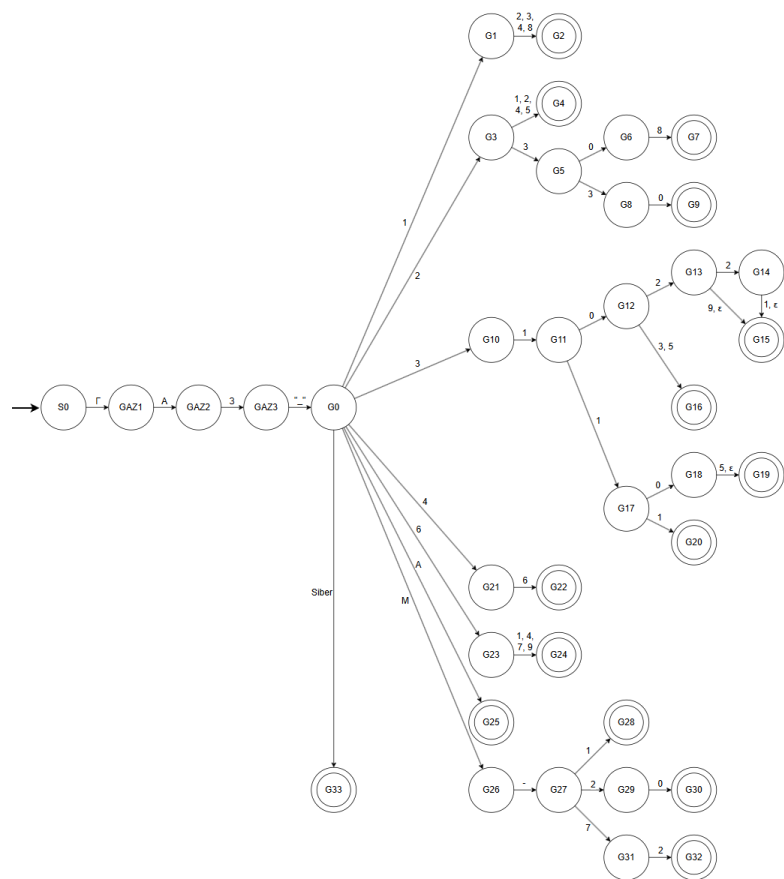


Рис. 3. НДКА для ГАЗ

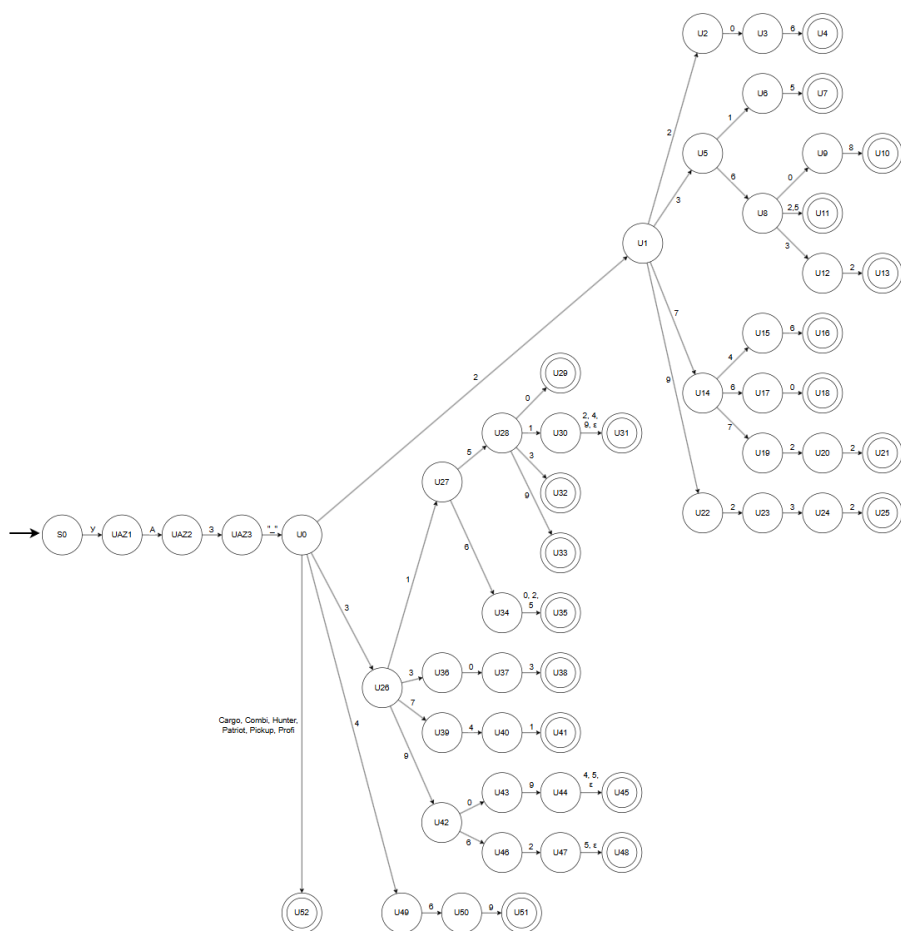


Рис. 4. НДКА для УАЗ

Далее созданные НДКА были объединены в один НДКА с начальным состоянием S_0 . Из начального состояния существует 5 переходов, в зависимости от которых выбирается марка, корректность модели автомобиля которой будет проверяться:

- «В» или «L» – ВАЗ (LADA);
- «У» – УАЗ;
- «Г» – ГАЗ;
- «М» – Москвич.

Полученный недетерминированный конечный автомат представлен на рисунке 5.

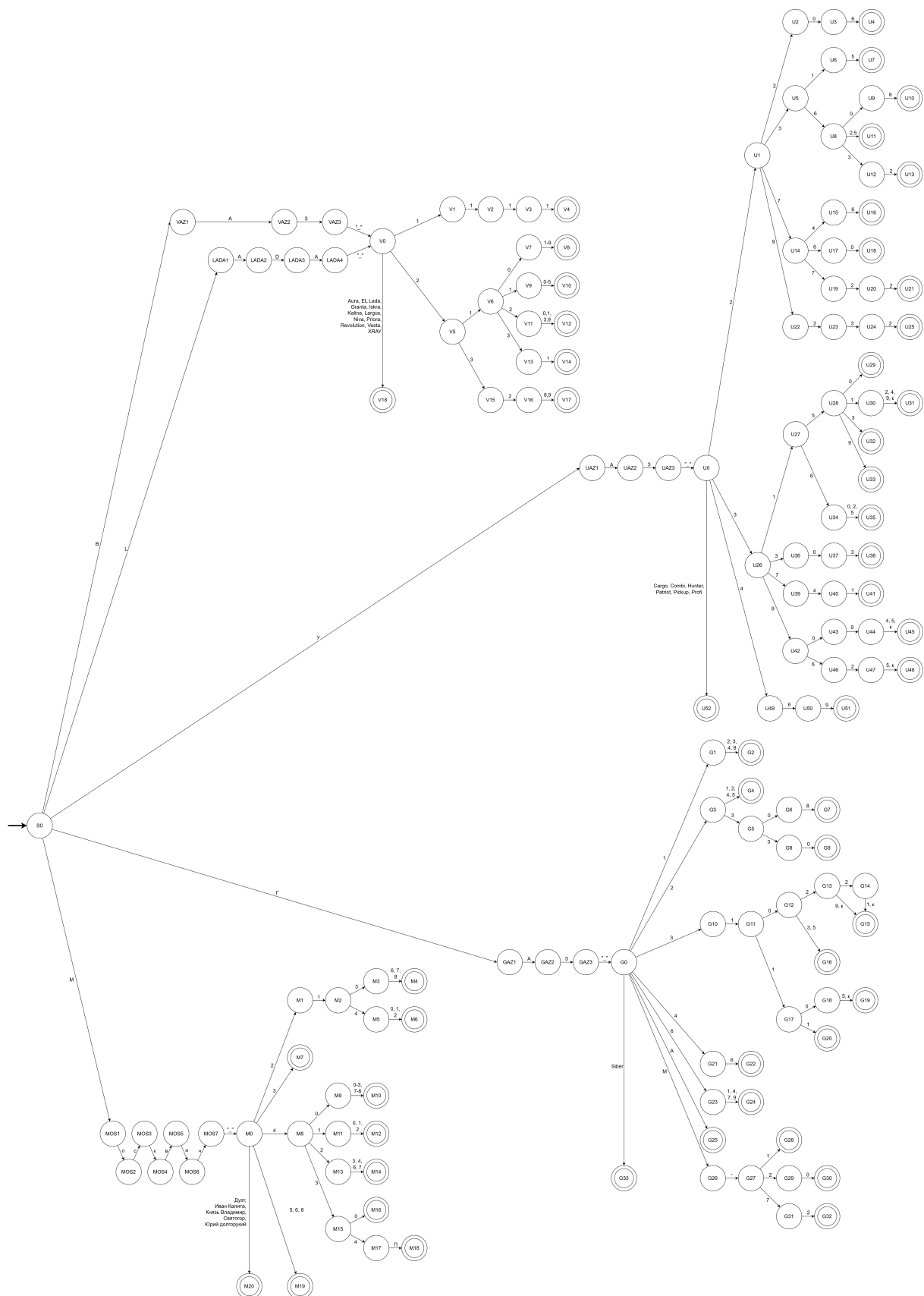


Рис. 5. НДКА для распознавания модели автомобиля

1.5 Детерминированный конечный автомат

Полученный детерминированный конечный автомат для рассматриваемого регулярного выражения представлен на рисунке 6.

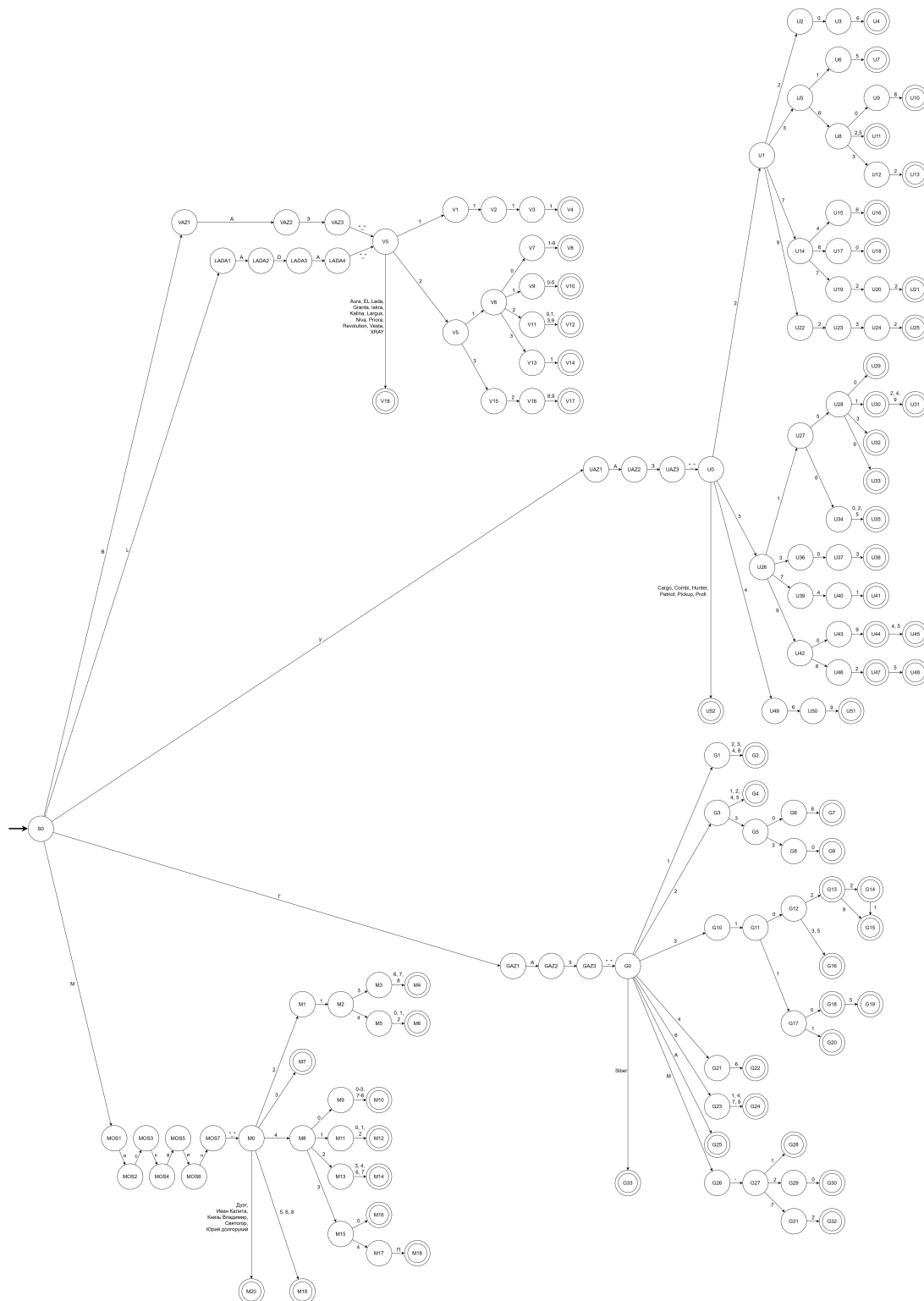


Рис. 6. НДКА для распознавания модели автомобиля

В таблице 1 представлены переходы разработанного детерминированного конечного автомата.

Таблица 1. Переходы детерминированного конечного автомата

Текущее состояние	Входящий символ	Следующее состояние
S0	B	VAZ1
S0	L	LADA1
S0	У	UAZ1
S0	Г	GAZ1
S0	M	MOS1
VAZ1	A	VAZ2
VAZ2	3	VAZ3
VAZ3	« »	V0
LADA1	A	LADA2
LADA2	D	LADA3
LADA3	A	LADA4
LADA4	« »	V0
V0	Aura, Granta, Iskra, Kalina, Largus, Niva, Priora, Revolution, Vesta, XRAY	V18
V0	1	V1
V1	1	V2
V2	1	V3
V3	1	V4
V5	1	V6
V5	3	V15
V6	0	V7
V6	1	V9
V6	2	V11
V6	3	V13
V7	1-9	V8
V9	0-5	V10
V11	0, 1, 3, 9	V12
V13	1	V14
V15	2	V16
V16	8, 9	V17

Продолжение на следующей странице

Текущее состояние	Входящий символ	Следующее состояние
UAZ1	A	UAZ2
UAZ2	3	UAZ3
UAZ3	« »	U0
U0	Cargo, Combi, Hunter, Patriot, Pickup, Profi	U52
U0	2	U1
U0	3	U26
U0	4	U49
U1	2	U2
U1	3	U5
U1	7	U14
U1	9	U22
U2	0	U3
U3	6	U4
U5	1	U6
U5	6	U8
U6	5	U7
U8	0	U9
U8	2, 5	U11
U8	3	U12
U9	8	U10
U12	2	U13
U14	5	U15
U14	6	U17
U14	7	U19
U15	6	U16
U17	0	U18
U19	2	U20
U20	2	U21
U22	2	U23
U23	3	U24
U24	2	U25
U26	1	U27
U26	3	U36

Продолжение на следующей странице

Текущее состояние	Входящий символ	Следующее состояние
U26	7	U39
U26	9	U42
U27	5	U28
U27	6	U34
U28	0	U29
U28	1	U30
U28	3	U32
U28	9	U33
U30	2, 4, 9	U31
U36	0	U37
U37	3	U38
U39	4	U40
U40	1	U41
U42	0	U43
U42	6	U46
U43	9	U44
U44	4, 5	U45
U46	2	U47
U47	5	U48
U49	6	U50
U50	9	U51
GAZ1	A	GAZ2
GAZ2	Z	GAZ3
GAZ3	« »	G0
G0	Siber	G33
G0	1	G1
G0	2	G3
G0	3	G10
G0	4	G21
G0	6	G23
G0	A	G25
G0	M	G26
G1	2, 3, 4, 8	G2

Продолжение на следующей странице

Текущее состояние	Входящий символ	Следующее состояние
G3	1, 2, 4, 5	G4
G3	3	G5
G5	0	G6
G5	3	G8
G6	8	G7
G8	0	G9
G10	1	G11
G11	0	G12
G11	1	G17
G12	2	G13
G12	3, 5	G16
G13	2	G14
G13	9	G15
G14	1	G15
G17	0	G18
G18	5	G19
G21	6	G22
G23	1, 4, 7, 9	G24
G26	-	G27
G27	1	G28
G27	2	G29
G27	7	G31
G29	0	G30
G31	2	G32
MOS1	о	MOS2
MOS2	с	MOS3
MOS3	к	MOS4
MOS4	в	MOS5
MOS5	и	MOS6
MOS6	ч	MOS7
MOS7	« »	M0
M0	Дуэт, Иван Калита, Князь Влади- мир, Святогор, Юрий Долгорукий	M20
M0	2	M1

Продолжение на следующей странице

Текущее состояние	Входящий символ	Следующее состояние
M0	3	M7
M0	4	M8
M0	5, 6, 8	M19
M1	1	M2
M2	3	M3
M2	4	M5
M3	6, 7, 8	M4
M5	0, 1, 2	M6
M8	0	M9
M8	1	M11
M8	2	M13
M8	3	M15
M9	0-3, 7, 8	M10
M11	0, 1, 2	M12
M13	3, 4, 6, 7	M14
M15	0	M16
M15	4	M17
M17	Π	M18

2 Особенности реализации

2.1 Структуры данных

Для данной лабораторной работы была разработана модуль, который позволяет:

- Создавать детерминированные конечные автоматы с помощью:
 - матрицы переходов и матрицы выходов;
 - перечисления узлов с указанием их дочерних узлов;
 - композиции уже существующих КА;
- обрабатывать данные с помощью КА;
- генерировать случайные корректные данные подходящие под созданный КА.

Реализация конечного автомата выполнена с использованием шаблонов, что позволяет, используя один и тот же интерфейс, обрабатывать не только строки, но и любой перечисляемый тип данных, например вектора или массивы с любыми внутренними типами данных.

Также следует отметить, что выходами при переходах между состояниями КА являются выходы функций, а не константные значения, что позволяет при необходимости расширить создаваемый КА до КА с магазинной памятью. При создании КА также можно определить функцию-обработчик, которая будет обрабатывать выходы данных функций. С помощью изменения функции-обработчика можно реализовать логирование работы КА, определение символа на котором КА прекратил работу, подсчёт частоты появления символов во входных данных и т.д..

Для генерации случайных данных можно использовать уже созданный автомат. Программа позволяет автоматически найти для каждого перехода подходящие элементы из заданного алфавита с помощью одного обхода графа КА. Во время генерации данных можно указать вероятность с которой сгенерированные корректные данные будут случайным образом повреждены.

2.1.1 Класс Action

Данный класс реализует хранение действий при переходах между состояниями КА. Класс Action является обёрткой над обычной функцией с заданной сигнатурой. Использование данного класса позволяет совместно хранить и функции и Null значения, что полезно, если не все переходы в КА должны иметь выход. Код данного класса представлен на листинге 1.

Листинг 1. Код класса Action

```
1 template<typename Signature>
2 class Action {
3 private:
4     std::function<Signature> func;
5 }
```

```

6 public:
7     Action() = default;
8     Action(const Action& other) = default;
9
10    Action(Action&& other) noexcept = default;
11
12    template<typename F>
13    requires (!std::is_same_v<std::decay_t<F>, Action<Signature>> && std
        ::is_constructible_v<std::function<Signature>, F&&>)
14    Action(F&& other);
15
16    Action& operator=(Action&& other) noexcept;
17
18    template<typename... Args>
19    auto operator()(Args&&... args) const;
20
21    explicit operator bool() const;
22
23    void reset();
24 };

```

Для данного класса был переопределён оператор вызова, который позволяет вызывая экземпляр данного класса выполнить функцию внутри него. Данный метод был объявлен шаблонным, чтобы сделать возможным вызов функций с разными сигнатурами.

Вход: аргументы для вызова внутренней функции.

Выход: выполненная внутренняя функция с заданными аргументами.

Код переопределения оператора вызова представлен на листинге 2.

Листинг 2. Код переопределения оператора вызова

```

1 Action& Action::operator=(Action&& other) noexcept {
2     if (this != &other) {
3         func = std::move(other.func);
4     }
5     return *this;
6 }

```

2.1.2 Класс FSM

Класс FSM реализует создание, хранение и использование конечных автоматов. Данный класс также объявлен шаблонным, что позволяет использовать один класс для работы с различными итерируемыми типами данных. В качестве шаблонов указывается тип входных данных, например строка или вектор чисел, и тип выходных данных при выполнении действий при переходах между узлами, например строка, символ, экземпляр какого-либо класса или void в случае если выходы при переходах не предусмотрены. Код данного класса представлен на листинге 3

Листинг 3. Код класса FSM

```

1 template<typename Data, typename ActionOut>
2     requires std::ranges::range<Data>

```

```

3 class FSM {
4 private:
5
6     int currentState;
7
8     using ElementType = std::ranges::range_value_t<Data>;
9     using Condition = std::function<bool(ElementType)>;
10    using ActionPtr = Action<ActionOut(ElementType)>*;
11    using Harvester = std::function<void(ActionOut)>;
12
13    ElementType element;
14
15    std::deque<ElementType> dataProc;
16
17    std::vector<ActionPtr> matrixActions;
18    std::vector<Condition> matrixConditions;
19    std::vector<int> matrixStates;
20
21    std::vector<int> offsets;
22    std::vector<int> finalStates;
23
24    std::function<void(ActionOut)> outputsHarvester;
25
26    std::vector<std::vector<ElementType>> elemsConditions;
27
28    bool flagEnd = false;
29
30    bool doStep();
31
32    FSM(
33        int stateCount,
34        std::unordered_map<int, std::vector<std::tuple<int, Condition,
35            ActionPtr>>> transitions,
36        std::vector<int> finals,
37        std::function<void(ActionOut)> harvester
38    );
39 public:
40    FSM() = default;
41
42    FSM(
43        std::vector<std::vector<Condition>> conditions,
44        std::vector<std::vector<ActionPtr>> actions,
45        std::vector<int> finals,
46        std::function<void(ActionOut)> harvester = nullptr
47    );
48
49    using Transition = std::pair<int, std::pair<Condition, ActionPtr>>;
50    using TransitionMap = std::unordered_map<int, std::vector<Transition
51        >>;
52
53    FSM(
54        int stateCount,
55        TransitionMap transitions,
56        std::vector<int> finals,
57        std::function<void(ActionOut)> harvester = nullptr

```

```

57     );
58
59     using TransitionNull = std::pair<int, Condition>;
60     using TransitionMapNull = std::unordered_map<int, std::vector<
        TransitionNull>>;
61
62     FSM(
63         int stateCount,
64         TransitionMapNull transitions,
65         std::vector<int> finals,
66         ActionPtr allAction = nullptr,
67         std::function<void(ActionOut)> harvester = nullptr
68     );
69
70
71     FSMResult start(Data& data, int startState = 0, Data* alphabet =
        nullptr);
72
73     std::deque<ElementType> getRemains();
74
75     int getStateCount() const;
76
77     std::vector<std::tuple<int, Condition, ActionPtr>>
        getTransitionsFrom(int state) const;
78
79     bool isFinalState(int state) const;
80
81     std::vector<int> getFinalStates() const;
82
83     Harvester getHarvester() const;
84
85     void genElemsOfConditions(Data* alphabet);
86     Data genElem(std::mt19937& generator, Data* alphabet, float prop, int
        startState = 0);
87
88     template<typename D, typename AO>
89     friend FSM<D, AO> mergeFSM(
90         const FSM<D, AO>& fsm1, int state1,
91         const FSM<D, AO>& fsm2, int state2,
92         std::function<void(AO)> harvester
93     );
94 };

```

Назначение полей класса

- currentState – активный узел КА.
- element – текущий обрабатываемый элемент из входных данных.
- dataProc – входные данные, перезаписанные в двухстороннюю очередь, которая позволяет выполнять быстрое удаление значений из начала.

- `matrixActions` – плоский вектор, хранящий матрицу выходов. Используется плоский вектор для обеспечения наилучшей кэш-лояльности.
- `matrixConditions` – плоский вектор, хранящий матрицу переходов. В нём также используется плоский вектор.
- `matrixStates` – вектор, содержащий номера узлов, входящих в данный конечный автомат.
- `offsets` – вспомогательный вектор для определения в плоских векторах условий и действий, соответствующих определённому узлу.
- `finalStates` – вектор, хранящий номера финальных состояний.
- `outputsHarvester` – функция-обработчик выходов функций, использующихся при переходах. При любом заданном шаблоне возвращает `void`, так как использование данной функции предполагает использование захвата внешних переменных для хранения вычисляемых значений.
- `elemsConditions` – двумерный вектор для хранения вычисленных допустимых элементов из входного алфавита для каждого узла.

Для класса `FSM` было реализовано три конструктора с разными входами.

Первый конструктор является наиболее сложным для ручного использования, так как требует записи матриц переходов и выходов. При использовании данного конструктора заданные матрицы просто копируются в поля данного экземпляра. Код данного конструктора представлен на листинге 4.

Вход: матрицы переходов и выходов, вектор финальных состояний, функцию-обработчик выходов.

Выход: созданный конечный автомат по заданным матрицам.

Листинг 4. Код первого конструктора класса `FSM`

```

1 FSM(
2     std::vector<std::vector<Condition>> conditions,
3     std::vector<std::vector<ActionPtr>> actions,
4     std::vector<int> finals,
5     std::function<void(ActionOut)> harvester = nullptr
6 ) : outputsHarvester(harvester)
7 {
8     if (actions.size() == conditions.size()) {
9         finalStates = finals;
10        int stateCount = actions.size();
11        offsets.reserve(stateCount + 1);
12        offsets.push_back(0);
13
14        for (int i = 0; i < stateCount; i++) {
15            for (int j = 0; j < stateCount; j++) {
16                if (conditions[i][j]) {
17                    matrixActions.push_back(actions[i][j]);
18                    matrixConditions.push_back(conditions[i][j]);
19                    matrixStates.push_back(j);
20                }
21            }
22        }
23    }
24 }

```

```

22         offsets.push_back(matrixActions.size());
23     }
24 }
25 }

```

Второй и третий конструкторы являются более простыми способами создания конечных автоматов. При их использовании необходимо перечислить для каждого узла узлы КА, в которые возможен переход, условие перехода и действие при переходе. Третий конструктор отличается от второго тем, что не требует записи действия при переходе для каждого задаваемого перехода, а позволяет задать одно действие для всех переходов (например, функция, возвращающая полученный элемент). Коды данных конструкторов представлены на листинге 5.

Вход для второго конструктора: количество узлов в КА, словарь, содержащий перечисление узлов, условий перехода и действий при переходе для каждого узла КА, вектор финальных состояний и функция-обработчик выходов.

Вход для третьего конструктора: количество узлов в КА, словарь, содержащий перечисление узлов и условий перехода для каждого узла КА, вектор финальных состояний, действие при любом переходе и функция-обработчик выходов.

Выход: созданный конечный автомат с заданными узлами и переходами между ними.

Листинг 5. Код второго и третьего конструкторов класса FSM

```

1 using Transition = std::pair<int, std::pair<Condition, ActionPtr>>;
2 using TransitionMap = std::unordered_map<int, std::vector<Transition>>;
3
4 FSM(
5     int stateCount,
6     TransitionMap transitions,
7     std::vector<int> finals,
8     std::function<void(ActionOut)> harvester = nullptr
9 ) : outputsHarvester(harvester)
10 {
11     finalStates = std::move(finals);
12     offsets.resize(stateCount + 1);
13
14     for (int state = 0; state < stateCount; ++state) {
15         offsets[state] = static_cast<int>(matrixActions.size());
16
17         auto it = transitions.find(state);
18         if (it != transitions.end()) {
19             for (const auto& [to, condAct] : it->second) {
20                 auto [condition, action] = condAct;
21                 if (condition) {
22                     matrixActions.push_back(action);
23                     matrixConditions.push_back(condition);
24                     matrixStates.push_back(to);
25                 }
26             }
27         }
28     }
29     offsets[stateCount] = static_cast<int>(matrixActions.size());
30 }

```

```

31
32 using TransitionNull = std::pair<int, Condition>;
33 using TransitionMapNull = std::unordered_map<int, std::vector<
    TransitionNull>>;
34
35 FSM(
36     int stateCount,
37     TransitionMapNull transitions,
38     std::vector<int> finals,
39     ActionPtr allAction = nullptr,
40     std::function<void(ActionOut)> harvester = nullptr
41 ) : outputsHarvester(harvester)
42 {
43     finalStates = std::move(finals);
44     offsets.resize(stateCount + 1);
45
46     for (int state = 0; state < stateCount; ++state) {
47         offsets[state] = static_cast<int>(matrixActions.size());
48
49         auto it = transitions.find(state);
50         if (it != transitions.end()) {
51             for (const auto& [to, cond] : it->second) {
52                 auto condition = cond;
53                 if (condition) {
54                     matrixActions.push_back(allAction);
55                     matrixConditions.push_back(condition);
56                     matrixStates.push_back(to);
57                 }
58             }
59         }
60     }
61     offsets[stateCount] = static_cast<int>(matrixActions.size());
62 }

```

Для перехода в следующий узел КА при заданных входных данных используется приватный метод doStep. Данный метод берёт следующий элемент из входных данных и проверяет возможность перехода в каждый следующий узел из текущего. Если существует подходящий переход, выполняется действие при переходе и вызывается функция обработчик выходов при переходах, а после меняется номер текущего состояния и возвращается значение true. Если переход не был найден, возвращается false. Выходные значения данного метода обрабатываются в методе start. Код метода doStep представлен на листинге 6.

Вход: текущее активное состояние в КА, входные данные, матрица переходов и действий при переходах, функция-обработчик выходов функций при переходах.

Выход: изменённое текущее активное состояние в КА и выполненное действие при переходе или изменённый флаг достижения финального состояния.

Листинг 6. Код метода doStep

```

1 bool doStep() {
2     int condStart = offsets[currentState];
3     int condEnd = offsets[currentState + 1];
4
5     if (dataProc.size() > 0) {

```

```

6         element = dataProc.front();
7         dataProc.pop_front();
8     }
9     else {
10         if (std::count(finalStates.begin(), finalStates.end(),
11             currentState) > 0) {
12             flagEnd = true;
13             return false;
14         }
15
16         for (int i = condStart; i < condEnd; i++) {
17             if (matrixConditions[i](element)) {
18                 if (matrixActions[i]) {
19                     ActionOut res = (*matrixActions[i])(element);
20                     if (outputsHarvester) {
21                         outputsHarvester(res);
22                     }
23                 }
24                 currentState = matrixStates[i];
25
26                 return true;
27             }
28         }
29         return false;
30     };

```

Для запуска обработки входных данных созданным КА необходимо использовать метод `start`. Данный метод перезаписывает входные данные в двухстороннюю очередь и последовательно выполняет переход к следующему состоянию с помощью метода `doStep`. Если больше нет доступных переходов или входные данные закончились, начинается обработка результатов, в течении которой определяется из-за чего завершилась работа КА: данные соответствуют (было достигнуто финальной состояние и входные данные закончились), был найден элемент не входящий в заданный алфавит или данные не соответствуют. Код данного метода можно увидеть на листинге 7.

Вход: входные данные, начальное состояние КА, алфавит входного языка.

Выход: выполненные действия при переходах, соответствующих входным данным и результат обработки данных.

Листинг 7. Код метода `start`

```

1 FSMResult start(Data& data, int startState = 0, Data* alphabet = nullptr
2     ) {
3     if (data.empty()) {
4         return FSMResult::Mismatch;
5     }
6     if (startState < offsets.size() - 1) {
7         dataProc.clear();
8         for (const ElementType& element : data) {
9             dataProc.push_back(element);
10        }
11        currentState = startState;

```



```

12     flagEnd = false;
13     while (doStep()) { }
14     if (flagEnd) {
15         return FSMResult::Match;
16     }
17     if (alphabet) {
18         for (ElementType alphElem : *alphabet) {
19             if (alphElem == element) {
20                 return FSMResult::Mismatch;
21             }
22         }
23         return FSMResult::OutOfAlphabet;
24     }
25     return FSMResult::Mismatch;
26 };

```

Для хранения результата обработки входных данных был создан класс-перечисление FSMResult, который имеет три допустимых значения: подходит, не подходит, не из алфавита. Код определения данного перечисления представлен на листинге 8.

Листинг 8. Код класса-перечисления FSMResult

```

1 enum class FSMResult {
2     Match,
3     Mismatch,
4     OutOfAlphabet
5 };

```

Для данного перечисления была создана функция resToStr, которая принимает его экземпляр и возвращает строку для вывода пользователю, соответствующую значению экземпляра. Код данной функции представлен на листинге 9.

Вход: результат обработки данных конечным автоматом.

Выход: строка, соответствующая полученному результату.

Листинг 9. Код функции resToStr

```

1 wstring resToStr(FSMResult result) {
2     switch (result) {
3         case FSMResult::Match: return L"Строка соответствует";
4         case FSMResult::Mismatch: return L"Строка не соответствует";
5         case FSMResult::OutOfAlphabet: return L"Используется символ не из алфавита";
6     }
7 }

```

Для построения композиции конечных автоматов была создана функция mergeFSM, которая «склеивает» заданные состояния двух КА. Данная функция создаёт новый экземпляр КА, в котором будет содержаться $n + m - 1$ состояний, где n и m – количество узлов в первом и во втором КА соответственно. При этом следует отметить, что нумерация состояний изменяется – состояния из первого КА остаются со своими номерами, а номер каждого состояния из второго КА сдвигается, так чтобы являться минимальным номером, не пересекающимся с номерами состояния из первого КА. Код функции mergeFSM представлен на листинге 10.

Вход: два КА и два номера состояний из первого и второго КА, которые необходимо «склеить» между собой.

Выход: новый конечный автомат, состоящий из узлов первого и второго КА так, что заданные состояния преобразованы в одно состояние.

Листинг 10. Код функции mergeFSM

```
1 FSM<Data, ActionOut> mergeFSM(  
2     const FSM<Data, ActionOut>& fsm1, int state1,  
3     const FSM<Data, ActionOut>& fsm2, int state2,  
4     std::function<void(ActionOut)> harvester = nullptr  
5 ) {  
6     using Condition = typename FSM<Data, ActionOut>::Condition;  
7     using ActionPtr = typename FSM<Data, ActionOut>::ActionPtr;  
8  
9     int n1 = fsm1.getStateCount();  
10    int n2 = fsm2.getStateCount();  
11  
12    int totalStates = n1 + n2 - 1;  
13  
14    std::unordered_map<int, std::vector<std::tuple<int, Condition,  
15        ActionPtr>>> transitions;  
16  
17    for (int s = 0; s < n1; s++) {  
18        auto trans = fsm1.getTransitionsFrom(s);  
19        for (const auto& [to, cond, act] : trans) {  
20            transitions[s].emplace_back(to, cond, act);  
21        }  
22    }  
23  
24    auto remapState = [&](int s) -> int {  
25        if (s == state2) {  
26            return state1;  
27        }  
28        else if (s < state2) {  
29            return s + n1;  
30        }  
31        else {  
32            return s + n1 - 1;  
33        }  
34    };  
35  
36    for (int s = 0; s < n2; s++) {  
37        auto trans = fsm2.getTransitionsFrom(s);  
38        int fromRemapped = remapState(s);  
39  
40        for (const auto& [to, cond, act] : trans) {  
41            int toRemapped = remapState(to);  
42            transitions[fromRemapped].emplace_back(toRemapped, cond, act  
43                );  
44        }  
45    }  
46  
47    std::vector<int> finalStates;
```

```

48     for (int fs : fsm1.getFinalStates()) {
49         finalStates.push_back(fs);
50     }
51
52     for (int fs : fsm2.getFinalStates()) {
53         int remapped = remapState(fs);
54         if (std::find(finalStates.begin(), finalStates.end(), remapped)
55             == finalStates.end()) {
56             finalStates.push_back(remapped);
57         }
58     }
59     return FSM<Data, ActionOut>(totalStates, std::move(transitions), std
        ::move(finalStates), harvester);

```

Для подготовки КА к генерации корректных входных данных используется метод `genElemOfConditions`, которая производит полный обход графа данного КА. В ходе обхода для каждого узла вычисляются элементы из входного алфавита, которые позволяют совершить переход в один из следующих узлов. Найденные элементы записываются в соответствующую позицию двумерного вектора `elemsConditions`. Вызывать данный метод необходимо только в случае, если был изменён входной алфавит или предикаты переходов в узлы, иначе вызов метода никак не повлияет на дальнейшую генерацию корректных данных. Код данного метода представлен на листинге 11.

Вход: входной алфавит и матрица переходов КА.

Выход: вычисленные элементы входного алфавита, позволяющие выполнить переход, для каждого узла.

Листинг 11. Код метода `genElemOfConditions`

```

1 void genElemOfConditions(Data* alphabet) {
2     for (int i = 0; i < matrixConditions.size(); i++) {
3         elemsConditions.push_back(std::vector<ElementType>());
4         for (ElementType elem : *alphabet) {
5             if (matrixConditions[i](elem)) {
6                 elemsConditions[i].push_back(elem);
7             }
8         }
9     }
10 }
11 }

```

После вызова метода `genElemOfConditions`, можно начинать генерацию корректных входных данных. Для этого используется метод `genElem`, который случайным образом формирует путь по графу КА. При переходе в новый узел случайным образом выбирается элемент, позволяющий выполнить переход, полученный элемент добавляется к результирующей переменной и производится переход в следующее состояние, соответствующее выбранному элементу. При вызове данного метода можно указать вероятность специальной «порчи» корректных данных – если в случайное число оказывается меньше заданной вероятности, то случайный элемент из результирующей переменной заменяется на случайный элемент из входного алфавита. Код данного метода представлен на листинге 12.

Вход: генератор случайных чисел, входной алфавит, вероятность «порчи данных» и начальное состояние из КА при генерации.

Выход: случайным образом сформированные входные данные, которые могут оказаться испорчены с заданной вероятностью.

Листинг 12. Код метода genElem

```
1 Data genElem(std::mt19937& generator, Data* alphabet, float prop, int
  startState = 0) {
2     if (elemsConditions.empty()) {
3         return Data();
4     }
5     currentState = startState;
6
7     Data res;
8
9     while (true) {
10        int condStart = offsets[currentState];
11        int condEnd = offsets[currentState + 1];
12        if (condStart == condEnd) {
13            break;
14        }
15        if (std::count(finalStates.begin(), finalStates.end(),
            currentState) > 0) {
16            std::uniform_int_distribution<> dist0(0, 1);
17            if (dist0(generator) == 1) {
18                break;
19            }
20        }
21
22        std::uniform_int_distribution<> dist1(condStart, condEnd - 1);
23        int i = dist1(generator);
24        std::uniform_int_distribution<> dist2(0, elemsConditions[i].size
            () - 1);
25        int j = dist2(generator);
26
27        res.push_back(elemsConditions[i][j]);
28        currentState = matrixStates[i];
29    }
30
31    std::uniform_real_distribution<double> dist(0.0, 1.0);
32    if (dist(generator) <= prop) {
33        std::uniform_int_distribution<> dist1(0, res.size() - 1);
34        std::uniform_int_distribution<> dist2(0, alphabet->size());
35        res[dist1(generator)] = (*alphabet)[dist2(generator)];
36    }
37
38    return res;
39 };
```

Также для упрощения создания конечных автоматов, обрабатывающих строки, была создана функция fsmFromWstr. Эта функция создаёт конечный автомат, который принимает только заданную строку. Для создания КА создаётся матрица действий при переходах, содержащая заданное действие для каждого перехода, и диагональная матрица переходов со сдвигом значений вправо. Код данной функции

представлен на листинге 13.

Вход: строка, которую должен принимать КА, действие выполняемое при переходах и функция-обработчик выходов функций.

Выход: экземпляр конечного автомата, который принимает заданную строку.

Листинг 13. Код функции fsmFromWstr

```
1 FSM<std::wstring, wchar_t> fsmFromWstr(std::wstring data, Action<wchar_t  
  (wchar_t)>* allAction = nullptr, std::function<void(wchar_t)>  
  harvester = nullptr) {  
2   int n = data.size();  
3   std::vector<std::vector<std::function<bool(wchar_t)>>> conditions(n  
    + 1, std::vector<std::function<bool(wchar_t)>>(n + 1, nullptr));  
4   std::vector<std::vector<Action<wchar_t(wchar_t)>>> actions(n + 1,  
    std::vector<Action<wchar_t(wchar_t)>>(n + 1, allAction));  
5   std::vector<int> finals({n});  
6  
7   for (int i = 0; i < n; i++) {  
8       conditions[i][i+1] = [value = data[i]](wchar_t c) {return c ==  
        value; };  
9   }  
10  
11   FSM<std::wstring, wchar_t> result(conditions, actions, finals,  
    harvester);  
12  
13   return result;  
14 }
```

2.2 Функция обработки пользовательского ввода

Для взаимодействия пользователя с программой была реализована функция userIO, которая предлагает пользователю варианты действий и выполняет действие исходя из ввода пользователя.

Вход: экземпляр КА с входными данными типа wstring, а выходными данными типа wchar_t, и строки, формирующиеся с помощью пользовательского ввода.

Выход: вывод меню пользователю и выполнение действий выбранных пользователем.

Данная функция представляет следующие меню:

1. **Главное меню:** пользователю предлагается выполнить одно из следующих действий:

- ввести строку;
- сгенерировать строки;
- сменить реализацию программы (КА или регулярное выражение);
- выйти из программы.

2. **Ввод строки.** У пользователя запрашивается ввод строки требующей проверки на соответствие регулярному выражению. После правильного ввода, программа обрабатывает полученную строку и выводит информацию о результате обработки.
3. **Генерация строк.** У пользователя запрашивается требуемое количество строк и вероятность «порчи» корректной строки. После правильного ввода программа генерирует заданной количество строк, обрабатывает их и выводит сводку о результатах обработки (количество соответствующих, не соответствующих и имеющих символы не из алфавита).

При необходимости программа может вывести все обработанные строки с цветовым выделением принятой и отброшенной частей строки.

Код данной функции представлен на листинге 14.

Листинг 14. Код функции userIO

```

1 void userIO(FSM<wstring, wchar_t> fsm) {
2     while (true) {
3         wcout << L"\nВыберите действие:\n\t[1] Ввести строку\n\t[2] Сгенерировать строки\n\t[3] Реализация(" << realization(useRegex)
4         << L")\n\t[e] Выход" << endl;
5         wstring input;
6         getline(wcin, input);
7
8         vector<wstring> vecStr;
9
10        if (input == L"1") {
11            wcout << L"Введите строку: ";
12            wstring data;
13            getline(wcin, data);
14            vecStr.push_back(data);
15        }
16        else if (input == L"2") {
17            int count = 0;
18            float prop = 0;
19            while (true) {
20                wcout << L"Введите кол-во строк: ";
21                string data;
22
23                getline(cin, data);
24                auto [ptr, ec] = std::from_chars(data.data(), data.data
25                    () + data.size(), count);
26
27                if (ec == std::errc{}) {
28                    if (count < 1 || count > 5000) {
29                        wcout << L"\nВведённое число выходит за диапазон
30                            [1, 5000]. Повторите попытку." << endl;
31                        continue;
32                    }
33                    break;
34                }
35                else if (ec == std::errc::invalid_argument) {

```

```

34         wcout << L"Неверный ввод. Повторите попытку." <<
35             endl;
36         continue;
37     }
38     else {
39         wcout << L"Введённое число выходит за диапазон [1, 5000]. Повторите попытку." << endl;
40         continue;
41     }
42 }
43 while (true) {
44     wcout << L"Введите вероятность случайной замены символа в строке: ";
45     string data;
46
47     getline(cin, data);
48     auto [ptr, ec] = std::from_chars(data.data(), data.data() + data.size(), prop);
49
50     if (ec == std::errc{} and ptr == data.data() + data.size()) {
51         if (prop < 0 || prop > 1) {
52             wcout << L"Введённое число выходит за диапазон [0, 1]. Повторите попытку." << endl;
53             continue;
54         }
55         break;
56     }
57     else if (ec == std::errc::invalid_argument) {
58         wcout << L"Неверный ввод. Повторите попытку." << endl;
59         continue;
60     }
61     else {
62         wcout << L"Введённое число выходит за диапазон [0, 1]. Повторите попытку." << endl;
63         continue;
64     }
65 }
66 for (int i = 0; i < count; i++) {
67     vecStr.push_back(fsm.genElem(gen, &alphabet, prop, 33));
68 }
69 wcout << L"Сгенерировано " << count << L" строк." << endl;
70 }
71 else if (input == L"3") {
72     useRegex = !useRegex;
73     continue;
74 }
75 else if (input == L"е") {
76     wcout << L"Выход из программы" << endl;
77     break;
78 }
79 else {
80     wcout << L"Неверный ввод. Повторите попытку." << endl;
81     continue;

```

```

82     }
83
84     FSMResult res;
85
86     wstring result;
87
88     int countSucc = 0;
89     int countFail = 0;
90
91     auto start = std::chrono::steady_clock::now();
92     for (wstring s : vecStr) {
93         if (useRegex) {
94             if (areAllCharsIn(s, alphabet)) {
95                 if (regex_match(s, re)) {
96                     res = FSMResult::Match;
97                     countSucc++;
98                     result += L"\033[48;2;0;70;0m" + s + L"\033[40m\033[48;2;0;70;0m" + resToStr(res) + L"\n";
99                 }
100             } else {
101                 res = FSMResult::Mismatch;
102                 countFail++;
103                 result += L"\033[48;2;200;0;0m" + s + L"\033[40m\033[48;2;200;0;0m" + resToStr(res) + L"\n";
104             }
105         } else {
106             res = FSMResult::OutOfAlphabet;
107             result += L"\033[48;2;200;0;0m" + s + L"\033[40m\033[48;2;200;0;0m" + resToStr(res) + L"\n";
108         }
109     }
110 }
111 else {
112     failureStr = s;
113     successStr.clear();
114     res = fsm.start(s, 33, &alphabet);
115
116     if (res == FSMResult::Match) {
117         countSucc++;
118     }
119     else if (res == FSMResult::Mismatch) {
120         countFail++;
121     }
122     result += L"\033[48;2;0;70;0m" + successStr + L"\033[48;2;200;0;0m" + failureStr + L"\033[40m\033[48;2;200;0;0m" + resToStr(res) + L"\n";
123 }
124 }
125 auto end = std::chrono::steady_clock::now();
126 std::chrono::duration<double, std::milli> time = end - start;
127
128 wcout << L"Строки обработаны за " << time.count() << L" мс.\n";
129 wcout << countSucc << L" строк соответствует" << endl;
130 wcout << countFail << L" строк не соответствует" << endl;
131 wcout << vecStr.size() - countSucc - countFail << L" строк имеют символы не из алфавита" << endl;

```



```

132         while (true) {
133             wcout << L"\nВывести_результаты_обработки?\n\t[1] _Да\n\t[2] _
                Нет" << endl;
134             getline(wcin, input);
135             if (input == L"1") {
136                 wcout << L"Обработанные_строки:" << endl;
137                 wcout << result << endl;
138                 break;
139             }
140             else if (input == L"2") {
141                 break;
142             }
143             else {
144                 wcout << L"Неверный_ввод._Повторите_попытку." << endl;
145                 continue;
146             }
147         }
148         vecStr.clear();
149     }
150 }

```

2.3 Создание конечного автомата соответствующего заданному регулярному выражению

Для создание конечного автомата, который сможет определять корректность введённых моделей автомобилей марок ВАЗ, УАЗ, ГАЗ, Москвич было создано 4 КА, каждый из которых реализует обработку моделей определённой марки. Каждый КА был создан с помощью перечисления узлов и следующих за ними узлов с их условиями переходов.

Для всех переходов в КА был задан один выход – функция `baseAction`, которая реализует базовое выходное действие – передачу полученного символа дальше в функцию-обработчик выходов.

Код данной функции представлен на листинге 15.

Вход: символ из входных данных.

Выход: символ, полученный во входе.

Листинг 15. Код функции `baseAction`

```

1 Action<wchar_t(wchar_t)> baseAction([](wchar_t c) { return c; });

```

Далее последовательно к созданным КА была применена функция `mergeFSM`, для создания композиции КА. При создании итоговой композиции КА была задана функция обработчик, которая разделяет входную строку на две части: принятая часть и отброшенная часть для формирования удобного вывода с использованием цветового выделения частей.

Код данной функции представлен на листинге 16.

Вход: символ из входной строки.

Выход: изменённые принятая и отброшенная части входной строки.

Листинг 16. Код класса harv_log

```
1 function<void(wchar_t)> harv_log([](wchar_t c) {  
2     successStr.push_back(c);  
3     failureStr.erase(0, 1);  
4     } );
```

Код задания состояний и переходов всех созданных конечных автоматов можно увидеть в приложении А.

3 Результаты работы программы

После запуска программы пользователю выводится главное меню (рисунок 7).

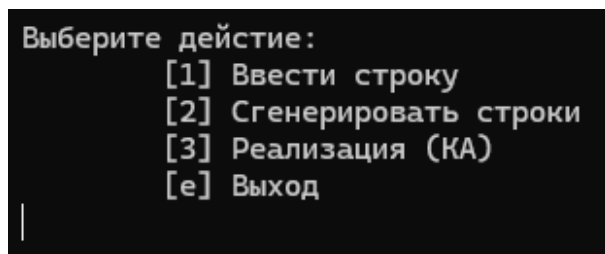
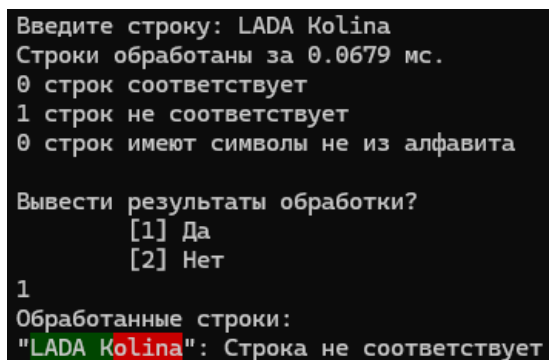
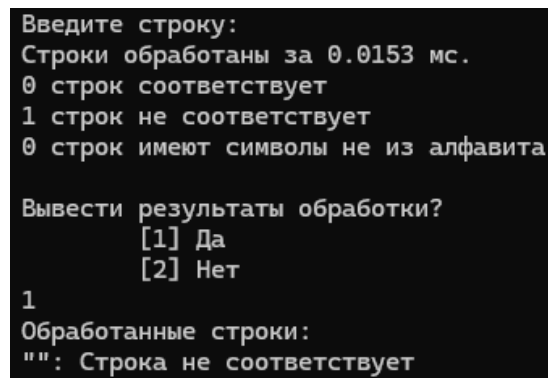


Рис. 7. Главное меню

После выбора действия «Ввести строку» и ввода строки, которую необходимо обработать, программа выводит сводку об обработке полученной строки и далее можно вывести более подробную информацию о введённой строке (рисунок 8).



(a) Несоответствующая строка



(b) Пустая строка

Рис. 8. Вывод информации об обработанной строке

Также пользователь может в главном меню выбрать пункт «Сгенерировать строки», после чего программа потребует от него ввода количества строк и вероятности случайной замены символа в корректной строке. Для количества строк определено ограничение в 5000, так как вывод в консоль большего количества строк будет очень долгим, а для вероятности заданы границы от 0 до 1 включительно. На рисунке 9 представлен ввод количества строк и вероятности случайной замены символа.

```

Введите кол-во строк: 9999
Введённое число выходит за диапазон [1, 5000]. Повторите попытку.
Введите кол-во строк: десять
Неверный ввод. Повторите попытку.
Введите кол-во строк: 10
Введите вероятность случайной замены символа в строке: 20
Введённое число выходит за диапазон [0, 1]. Повторите попытку.
Введите вероятность случайной замены символа в строке: 0.2

```

Рис. 9. Ввод количества строк и вероятности замены символа

После успешного ввода количества строк и вероятности, пользователю выводится краткая сводка об обработанных строках и по желанию пользователя выводится детальный вывод каждой строки (рисунок 10). В выводе подробной информации зелёным цветом выделяется часть строки, которую КА смог принять, а красным – не принятая часть.

```

Сгенерировано 10 строк.
Строки обработаны за 0.1652 мс.
9 строк соответствует
1 строк не соответствует
0 строк имеют символы не из алфавита

Вывести результаты обработки?
[1] Да
[2] Нет

1
Обработанные строки:
"УАЗ 29232": Строка соответствует
"LADA Priora": Строка соответствует
"УАЗ Cargo": Строка соответствует
"ГАЗ А": Строка соответствует
"Москвич Дуэт": Строка не соответствует
"УАЗ 3909": Строка соответствует
"ГАЗ 3103": Строка соответствует
"ГАЗ 69": Строка соответствует
"ГАЗ 2330": Строка соответствует
"ГАЗ 31022": Строка соответствует

```

Рис. 10. Вывод сводки и подробной информации об обработанных строках

Также пользователь может изменить реализацию программы выбрав вместо конечного автомата использование регулярного выражения с помощью модуля regex. При использовании реализации через КА в среднем строки обрабатываются в два раза быстрее, чем при использовании регулярного выражения. На рисунке 11 представлено сравнение скорости обработки одной и той же строки с использованием разных реализаций.

Следует отметить, что при использовании реализации через regex вывод подробной информации об обработанных строках упрощается, так как regex не поддерживает нахождение индекса символа, с которого началось несоответствие заданному регулярному выражению.

```

Выберите действие:
    [1] Ввести строку
    [2] Сгенерировать строки
    [3] Реализация (КА)
    [e] Выход
1
Введите строку: LADA Prioka
Строки обработаны за 0.0396 мс.
0 строк соответствует
1 строк не соответствует
0 строк имеют символы не из алфавита

Вывести результаты обработки?
    [1] Да
    [2] Нет
1
Обработанные строки:
"LADA Prioka": Строка не соответствует

```

(a) С использованием КА

```

Выберите действие:
    [1] Ввести строку
    [2] Сгенерировать строки
    [3] Реализация (regex)
    [e] Выход
1
Введите строку: LADA Prioka
Строки обработаны за 0.0953 мс.
0 строк соответствует
1 строк не соответствует
0 строк имеют символы не из алфавита

Вывести результаты обработки?
    [1] Да
    [2] Нет
1
Обработанные строки:
"LADA Prioka": Строка не соответствует

```

(b) С использованием regex

Рис. 11. Сравнение скорости обработки строки с помощью разных реализаций

Заключение

В ходе выполнения данной лабораторной работы было создано регулярное выражение, соответствующее моделям автомобилей марок ВАЗ, УАЗ, ГАЗ и Москвич. На основе полученного регулярного выражения были созданы недетерминированный КА и детерминированный КА, который состоит из 295 состояний.

Далее был разработан модуль для создания КА и работы с ними и с помощью него было реализовано программное представление составленного детерминированного конечного автомата.

Реализованная программа предоставляет пользователю следующие функции:

- обработка введенной пользователем строки;
- генерация до 5000 строк с задаваемой пропорцией несоответствующих строк и последующая обработка сгенерированных строк;
- смена реализации программы (КА или регулярное выражение);

Достоинства реализации

- Разработанный модуль позволяет реализовывать конечные автоматы, обрабатывающие не только строки, но и любые итерируемые данные. Также с помощью модуля можно создавать автоматы с магазинной памятью, что позволяет использовать данный модуль при создании распознавателей контекстно-свободных языков.
- Для генерации корректных строк, используется алгоритм, который автоматически для каждого перехода находит подходящие символы заданного входного алфавита. Асимптотическая сложность данного алгоритма составляет $O(q \cdot |\Sigma|)$, где q – количество переходов, а $|\Sigma|$ – количество символов во входном алфавите. Данный алгоритм запускается только один раз, после чего генерация строки происходит без лишних вычислений.
- Пользователю предоставляется возможность выбора реализации программы (использование модуля `regex` или использование КА).
- Реализована цветовое выделение принятой и не принятой частей входной строки.
- Разработанный КА в среднем работает более чем в два раза быстрее, чем соответствующее ему регулярное выражение в модуле `regex`.

Недостатки реализации

- Распределение вероятности генерации корректных строк неравномерное – наибольшая вероятность генерации у корректных строк, состоящих из наименьшего количества символов.

Масштабирование

- Добавить графический интерфейс для создания КА из созданного модуля или реализовать создание КА из файлов формата «.drawio», содержащих графическое описание КА.
- Реализовать преобразование регулярного выражения в КА из созданного модуля.

Список использованной литературы

- [1] Секция «Телематика». — Текст: электронный // tema.spbstu: [сайт]. — URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 29.03.2026).
- [2] Вирт, Н. Построение компиляторов / Н. Вирт. — М.: ДМК Пресс, 2010. — 190 с.
- [3] Ахо, А. В. Компиляторы: принципы, технологии и инструментарий [2-е изд.] / А. В. Ахо, М. С. Лам, Р. Сети, Дж. Д. Ульман ; Пер. с англ. — М. : ООО «И.Д. Вильямс», 2018. — 1184 с.

Приложение А. Исходный код создания конечных автоматов

```
1 wstring alphabet = L" _0123456789ACDEGHIKLMNPRSVXYabcdefghijklmnopstuvABГ  
   ДЗИКМПСУЮавгдзийклмнорстучьзя";  
2  
3 Action<wchar_t(wchar_t)> baseAction([](wchar_t c) { return c; });  
4  
5  
6 bool (*is0)(wchar_t) = [](wchar_t c) { return c == L'0'; };  
7 bool (*is1)(wchar_t) = [](wchar_t c) { return c == L'1'; };  
8 bool (*is2)(wchar_t) = [](wchar_t c) { return c == L'2'; };  
9 bool (*is3)(wchar_t) = [](wchar_t c) { return c == L'3'; };  
10 bool (*is4)(wchar_t) = [](wchar_t c) { return c == L'4'; };  
11 bool (*is5)(wchar_t) = [](wchar_t c) { return c == L'5'; };  
12 bool (*is6)(wchar_t) = [](wchar_t c) { return c == L'6'; };  
13 bool (*is7)(wchar_t) = [](wchar_t c) { return c == L'7'; };  
14 bool (*is8)(wchar_t) = [](wchar_t c) { return c == L'8'; };  
15 bool (*is9)(wchar_t) = [](wchar_t c) { return c == L'9'; };  
16  
17  
18 FSM<wstring, wchar_t>::TransitionMapNull tG;  
19 tG[0] = {  
20     {1, is1},  
21     {3, is2},  
22     {10, is3},  
23     {21, is4},  
24     {23, is6},  
25     {25, [](wchar_t c) { return c == L'A'; }},  
26     {26, [](wchar_t c) { return c == L'M'; }}  
27 };  
28 tG[1] = {  
29     {2, [](wchar_t c) { return c == L'2' || c == L'3' || c == L'4' || c  
        == L'8'; }},  
30 };  
31 tG[2] = {};  
32 tG[3] = {  
33     {4, [](wchar_t c) { return c == L'1' || c == L'2' || c == L'4' || c  
        == L'5'; }},  
34     {5, is3}  
35 };  
36 tG[4] = {};  
37 tG[5] = {  
38     {6, is0},  
39     {8, is3}  
40 };  
41 tG[6] = {  
42     {7, is8}  
43 };  
44 tG[7] = {};  
45 tG[8] = {  
46     {9, is0}  
47 };  
48 tG[9] = {};  
49 tG[10] = {
```

```

50     {11, is1}
51 };
52 tG[11] = {
53     {12, is0},
54     {17, is1}
55 };
56 tG[12] = {
57     {13, is2},
58     {16, [](wchar_t c) { return c == L'3' || c == L'5'; }}
59 };
60 tG[13] = {
61     {14, is2},
62     {15, is9}
63 };
64 tG[14] = {
65     {15, is1}
66 };
67 tG[15] = {};
68 tG[16] = {};
69 tG[17] = {
70     {18, is0},
71     {20, is1}
72 };
73 tG[18] = {
74     {19, is5}
75 };
76 tG[19] = {};
77 tG[20] = {};
78 tG[21] = {
79     {22, is6}
80 };
81 tG[22] = {};
82 tG[23] = {
83     {24, [](wchar_t c) { return c == L'1' || c == L'4' || c == L'7' || c
      == L'9'; }}
84 };
85 tG[24] = {};
86 tG[25] = {};
87 tG[26] = {
88     {27, [](wchar_t c) { return c == L'-' ; }}
89 };
90 tG[27] = {
91     {28, is1},
92     {29, is2},
93     {31, is7}
94 };
95 tG[28] = {};
96 tG[29] = {
97     {30, is0}
98 };
99 tG[30] = {};
100 tG[31] = {
101     {32, is2}
102 };
103 tG[32] = {};
104 tG[33] = { // start

```

```

105     {34, [](wchar_t c) { return c == L'Г'; }}
106 };
107 tG[34] = {
108     {35, [](wchar_t c) { return c == L'A'; }}
109 };
110 tG[35] = {
111     {36, [](wchar_t c) { return c == L'3'; }}
112 };
113 tG[36] = {
114     {0, [](wchar_t c) { return c == L'␣'; }}
115 };
116 FSM<wstring, wchar_t> fsmG(
117     37,
118     tG,
119     { 2, 4, 7, 9, 13, 14, 15, 16, 18, 19, 20, 22, 24, 25, 28, 30, 32 },
120     &baseAction,
121     nullptr
122 );
123 vector<wstring> strsG = { L"Siber" };
124 for (wstring s : strsG) {
125     fsmG = mergeFSM(fsmG, 0, fsmFromWstr(s, &baseAction), 0);
126 }
127
128
129 FSM<wstring, wchar_t>::TransitionMapNull tU;
130 tU[0] = {
131     {1, is2},
132     {26, is3},
133     {49, is4},
134     {56, [](wchar_t c) {return c == L'C'; }},
135     {57, [](wchar_t c) {return c == L'P'; }}
136 };
137 tU[1] = {
138     {2, is2},
139     {5, is3},
140     {14, is7},
141     {22, is9}
142 };
143 tU[2] = {
144     {3, is0}
145 };
146 tU[3] = {
147     {4, is6}
148 };
149 tU[4] = {};
150 tU[5] = {
151     {6, is1},
152     {8, is6}
153 };
154 tU[6] = {
155     {7, is5}
156 };
157 tU[7] = {};
158 tU[8] = {
159     {9, is0},
160     {11, [](wchar_t c) {return c == L'2' || c == L'5'; }},

```

```

161     {12, is3}
162 } ;
163 tU[9] = {
164     {10, is8}
165 } ;
166 tU[10] = {} ;
167 tU[11] = {} ;
168 tU[12] = {
169     {13, is2}
170 } ;
171 tU[13] = {} ;
172 tU[14] = {
173     {15, is4} ,
174     {17, is6} ,
175     {19, is7}
176 } ;
177 tU[15] = {
178     {16, is6}
179 } ;
180 tU[16] = {} ;
181 tU[17] = {
182     {18, is0}
183 } ;
184 tU[18] = {} ;
185 tU[19] = {
186     {20, is2}
187 } ;
188 tU[20] = {
189     {21, is2}
190 } ;
191 tU[21] = {} ;
192 tU[22] = {
193     {23, is2}
194 } ;
195 tU[23] = {
196     {24, is3}
197 } ;
198 tU[24] = {
199     {25, is2}
200 } ;
201 tU[25] = {} ;
202 tU[26] = {
203     {27, is1} ,
204     {36, is3} ,
205     {39, is7} ,
206     {42, is9}
207 } ;
208 tU[27] = {
209     {28, is5} ,
210     {34, is6}
211 } ;
212 tU[28] = {
213     {29, is0} ,
214     {30, is1} ,
215     {32, is3} ,
216     {33, is9}

```

```

217 } ;
218 tU[29] = {} ;
219 tU[30] = {
220     {31, [](wchar_t c) {return c == L'2' || c == L'4' || c == L'9'; }}
221 } ;
222 tU[31] = {} ;
223 tU[32] = {} ;
224 tU[33] = {} ;
225 tU[34] = {
226     {35, [](wchar_t c) {return c == L'0' || c == L'2' || c == L'5'; }}
227 } ;
228 tU[35] = {} ;
229 tU[36] = {
230     {37, is0}
231 } ;
232 tU[37] = {
233     {38, is3}
234 } ;
235 tU[38] = {} ;
236 tU[39] = {
237     {40, is4}
238 } ;
239 tU[40] = {
240     {41, is1}
241 } ;
242 tU[41] = {} ;
243 tU[42] = {
244     {43, is0} ,
245     {46, is6}
246 } ;
247 tU[43] = {
248     {44, is9}
249 } ;
250 tU[44] = {
251     {45, [](wchar_t c) {return c == L'4' || c == L'5'; }}
252 } ;
253 tU[45] = {} ;
254 tU[46] = {
255     {47, is2}
256 } ;
257 tU[47] = {
258     {48, is5}
259 } ;
260 tU[48] = {} ;
261 tU[49] = {
262     {50, is6}
263 } ;
264 tU[50] = {
265     {51, is9}
266 } ;
267 tU[51] = {} ;
268 tU[52] = { // start
269     {53, [](wchar_t c) {return c == L'y'; }}
270 } ;
271 tU[53] = {
272     {54, [](wchar_t c) {return c == L'A'; }}

```

```

273 };
274 tU[54] = {
275     {55, [](wchar_t c) {return c == L'3'; }}
276 };
277 tU[55] = {
278     {0, [](wchar_t c) {return c == L'␣'; }}
279 };
280 tU[56] = {};
281 tU[57] = {};
282 FSM<wstring, wchar_t> fsmU(
283     58,
284     tU,
285     { 4, 7, 10, 11, 13, 16, 18, 21, 25, 29, 30, 31, 32, 33, 35, 38, 41,
        44, 45, 47, 48, 51 },
286     & baseAction,
287     nullptr
288 );
289 fsmU = mergeFSM(fsmU, 0, fsmFromWstr(L"Hunter", &baseAction), 0);
290 vector<wstring> strsU1 = { L"argo", L"ombi" };
291 for (wstring s : strsU1) {
292     fsmU = mergeFSM(fsmU, 56, fsmFromWstr(s, &baseAction), 0);
293 }
294 vector<wstring> strsU2 = { L"atriot", L"ickup", L"rofi" };
295 for (wstring s : strsU2) {
296     fsmU = mergeFSM(fsmU, 57, fsmFromWstr(s, &baseAction), 0);
297 }
298
299
300 FSM<wstring, wchar_t>::TransitionMapNull tM;
301 tM[0] = {
302     {1, is2},
303     {7, is3},
304     {8, is4},
305     {19, [](wchar_t c) {return c == L'5' || c == L'6' || c == L'8'; }}
306 };
307 tM[1] = {
308     {2, is1}
309 };
310 tM[2] = {
311     {3, is3},
312     {5, is4}
313 };
314 tM[3] = {
315     {4, [](wchar_t c) {return c == L'6' || c == L'7' || c == L'8'; }}
316 };
317 tM[4] = {};
318 tM[5] = {
319     {6, [](wchar_t c) {return c == L'0' || c == L'1' || c == L'2'; }}
320 };
321 tM[6] = {};
322 tM[7] = {};
323 tM[8] = {
324     {9, is0},
325     {11, is1},
326     {13, is2},
327     {15, is3}

```

```

328 } ;
329 tM[9] = {
330     {10, [](wchar_t c) {return L'0' <= c && c <= L'3' || c == L'7' || c
        == L'8'; }}
331 } ;
332 tM[10] = {} ;
333 tM[11] = {
334     {12, [](wchar_t c) {return L'0' <= c && c <= L'2'; }}
335 } ;
336 tM[12] = {} ;
337 tM[13] = {
338     {14, [](wchar_t c) {return c == L'3' || c == L'4' || c == L'6' || c
        == L'7'; }}
339 } ;
340 tM[14] = {} ;
341 tM[15] = {
342     {16, is0} ,
343     {17, is4}
344 } ;
345 tM[16] = {} ;
346 tM[17] = {
347     {18, [](wchar_t c) {return c == L'П'; }}
348 } ;
349 tM[18] = {} ;
350 tM[19] = {} ;
351 tM[20] = { // start
352     {21, [](wchar_t c) {return c == L'М'; }}
353 } ;
354 tM[21] = {
355     {22, [](wchar_t c) {return c == L'о'; }}
356 } ;
357 tM[22] = {
358     {23, [](wchar_t c) {return c == L'с'; }}
359 } ;
360 tM[23] = {
361     {24, [](wchar_t c) {return c == L'к'; }}
362 } ;
363 tM[24] = {
364     {25, [](wchar_t c) {return c == L'в'; }}
365 } ;
366 tM[25] = {
367     {26, [](wchar_t c) {return c == L'и'; }}
368 } ;
369 tM[26] = {
370     {27, [](wchar_t c) {return c == L'ч'; }}
371 } ;
372 tM[27] = {
373     {0, [](wchar_t c) {return c == L'␣'; }}
374 } ;
375 FSM<wstring, wchar_t> fsmM(
376     28,
377     tM,
378     { 4, 6, 7, 10, 12, 14, 16, 18, 19 } ,
379     & baseAction,
380     nullptr
381 );

```

```

382 vector<wstring> strsM = { L"Дуэт", L"Иван_Калита", L"Князь_Владимир", L"
    Святогор", L"Юрий_Долгорукий" };
383 for (wstring s : strsM) {
384     fsmM = mergeFSM(fsmM, 0, fsmFromWstr(s, &baseAction), 0);
385 }
386
387
388 FSM<wstring, wchar_t>::TransitionMapNull tV;
389 tV[0] = {
390     {1, is1},
391     {5, is2}
392 };
393 tV[1] = {
394     {2, is1}
395 };
396 tV[2] = {
397     {3, is1}
398 };
399 tV[3] = {
400     {4, is1}
401 };
402 tV[4] = {};
403 tV[5] = {
404     {6, is1},
405     {15, is3}
406 };
407 tV[6] = {
408     {7, is0},
409     {9, is1},
410     {11, is2},
411     {13, is3}
412 };
413 tV[7] = {
414     {8, [](wchar_t c) {return L'0' <= c && c <= L'9';}}
415 };
416 tV[8] = {};
417 tV[9] = {
418     {10, [](wchar_t c) {return L'0' <= c && c <= L'5';}}
419 };
420 tV[10] = {};
421 tV[11] = {
422     {12, [](wchar_t c) {return c == L'0' || c == L'1' || c == L'3' || c
        == L'9';}}
423 };
424 tV[12] = {};
425 tV[13] = {
426     {14, is1}
427 };
428 tV[14] = {};
429 tV[15] = {
430     {16, is2}
431 };
432 tV[16] = {
433     {17, [](wchar_t c) {return c == L'8' || c == L'9';}}
434 };
435 tV[17] = {};

```



```

436 tV[18] = { // start
437     {19, [](wchar_t c) {return c == L'B'; }},
438     {22, [](wchar_t c) {return c == L'L'; }},
439 };
440 tV[19] = {
441     {20, [](wchar_t c) {return c == L'A'; }},
442 };
443 tV[20] = {
444     {21, [](wchar_t c) {return c == L'3'; }},
445 };
446 tV[21] = {
447     {0, [](wchar_t c) {return c == L'␣'; }},
448 };
449 tV[22] = {
450     {23, [](wchar_t c) {return c == L'A'; }},
451 };
452 tV[23] = {
453     {24, [](wchar_t c) {return c == L'D'; }},
454 };
455 tV[24] = {
456     {25, [](wchar_t c) {return c == L'A'; }},
457 };
458 tV[25] = {
459     {0, [](wchar_t c) {return c == L'␣'; }},
460 };
461 FSM<wstring, wchar_t> fsmV(
462     26,
463     tV,
464     { 4, 8, 10, 12, 14, 17 },
465     & baseAction,
466     nullptr
467 );
468 vector<wstring> strsV = { L"Aura", L"EL␣Lada", L"Granta", L"Iskra", L"
    Kalina", L"Largus", L"Niva", L"Priora", L"Revolution", L"Vesta", L"
    XRAY", };
469 for (wstring s : strsV) {
470     fsmV = mergeFSM(fsmV, 0, fsmFromWstr(s, &baseAction), 0);
471 }
472
473
474 function<void(wchar_t)> harv_log([](wchar_t c) {
475     successStr.push_back(c);
476     failureStr.erase(0, 1);
477 });
478
479 FSM<wstring, wchar_t> fsmAll = mergeFSM(
480     mergeFSM(fsmG, 33, fsmU, 52),
481     33,
482     mergeFSM(fsmM, 20, fsmV, 18),
483     20,
484     harv_log
485 );
486
487 fsmAll.genElementsOfConditions(&alphabet);
488
489 userIO(fsmAll);

```